

CUDA & Parallel Search

Asked 11 years, 5 months ago Modified 8 years, 6 months ago Viewed 1k times

1

I would like to do the following thing: Let's say that I have a sorted numbers vector of size N (N quite big) and a number x. I want to parallelize the searching of the right position for number x in this vector. For example:

myVector = [1, 2, 3, ..., 10000] and x = 3.2,

then I have to return 3. The first thread to find the right position should interrupt the job of others threads. Then the time to spend would be minimize :
t= min(t_1, t_2,....., t_number of threads) Do you think that using multithreading for looking for the right position could be faster? What about the communication between threads? Since once a value has been red by a thread and does not match the search, the others threads have to skip this value during the searching (maybe a boolean that has to be changed..

Do you have some advices to share regarding this algorithm?

parallel-processing cuda Edit tags

Share Edit Follow Close Flag

edited Mar 2, 2016 at 19:53

 talonmies

71.9k 35 200 282

asked Mar 7, 2013 at 8:47

 ALFRAM

193 2 14

- 5  Unless the sorted vector is already in the device memory it makes no sense to use CUDA for this. Binary search on the CPU has complexity log2 n. – RoBiK Mar 7, 2013 at 10:16
- 1  you may be interested in [thrust::lower_bound](#) or [thrust::partition_point](#) If you're not familiar with thrust, there is a [getting started guide](#). – Robert Crovella Mar 7, 2013 at 16:10

3 Answers

Sorted by:

Reset to default Date modified (newest first) 

0

Some time ago I wrote the following code that does similar thing:

```
#include "cuda_runtime.h"
#include "device_launch_parameters.h"

#include <stdio.h>
#include <stdlib.h>

__global__ void fast_finder(unsigned int *g_found, float x, float *y)
{
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int pos = (unsigned int)(x == y[i]);
    g_found[i * (1 - pos)] = i * pos;
}

int main(int argc, char *argv[])
{
    int N = 65536;
    unsigned int h_found, *d_found;
    float *h_y = (float *)malloc(N * sizeof(float)), *d_y, x = 5.0f;
    int nThreads = 1024, nBlocs = N / nThreads;

    for (int i = 0; i < N; ++i) h_y[i] = (float)(N - i - 1);

    if (x != h_y[0]) {
        cudaSetDevice(0);
        cudaMalloc((void **)&d_found, N * sizeof(unsigned int));
        cudaMalloc((void **)&d_y, N * sizeof(float));
        cudaMemcpy(d_y, h_y, N * sizeof(float), cudaMemcpyHostToDevice);

        fast_finder<<<nBlocs, nThreads>>>(d_found, x, d_y);
        cudaThreadSynchronize();

        cudaMemcpy(&h_found, d_found, sizeof(unsigned int),
        cudaMemcpyDeviceToHost);
        if (h_found) printf("%g found on %d. position!\n", x, h_found);
        else printf("%g not found!\n", x);

        cudaFree(d_y);
        cudaFree(d_found);

    } else printf("%g found on the first position!\n", x);

    free(h_y);

    getchar();
    return EXIT_SUCCESS;
}
```

Here each thread checks if the value provided by global thread index in `y` is equal to `x`. Should it be true, thread writes it's index to first position of the `g_found` array, else writes 0 to position of `g_found` provided by it's index. For `y` of length 16, containing value 5 on 11th position in `y` the out come is as follows:

```
g_found = { 10, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 }
```

In this case `y` need not to be sorted, but have to contain only unique values. This code can be easily altered to one finding (the device part) index where the provided `x` would be inserted, as follows:

```
__global__ void fast_finder(unsigned int *g_found, float x, float *y)
{
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int pos = (unsigned int)(x >= y[i] || x <= y[i+1]);
    g_found[i * (1 - pos)] = (i + 1) * pos;
}
```

Output of this version would be similar to mine. When `g_found` at position 0 is 0 then the value of `x` is not present in the array `y`. Whether the first element of `y` is equal to `x` is checked by host code, before the kernel is even called. It wouldn't be a problem to change this part too to apply

condition You want.

As You can see, in such a solution all threads work together and there is no need for any execution termination, provided x is found. Good thing would be also to apply packet search, meaning assign one thread to seek in a small subset of y , thus allowing y to be much bigger.

Share Edit Follow Flag

answered Mar 7, 2013 at 10:26



Dorota Kadłubowska
675 1 8 26



0



No need to communicate between threads and blocks. You can check to see if the value at the current index is greater than anticipated. If so return. Most threads will not survive this check.

Now you only have threads with indices where values are less than the expected value. Check to see if next value is greater or equal to the query and return the appropriate index.



Here is my untested kernel I am writing at 5 am.



```
template<typename ty>
__global__ static void search(int *out, ty *list, ty val, int n)
{
    int start = threadIdx.x + blockIdx.x * blockDim.x;
    for (int idx = start; idx < n; idx += gridDim.x * blockDim.x) {
        if (list[idx] >= val) return;
        ty next = list[idx + 1];
        if (idx == n-1 || next >= val) {
            *out = next == val ? (idx + 1) : idx;
            return;
        }
    }
}
```

That said, you really don't want to do this. You can get a worst case performance of $O(\log n)$ while using the CPU. That means searching a billion elements can be done in 32 steps. Unless you have the data already on the gpu and want to avoid a memory copy, this is much better off on the CPU.

Share Edit Follow Flag

answered Mar 7, 2013 at 10:25



Pavan Yalamanchili
12.1k 2 37 55



0



Ok I see. Thank you for your answers.



I am currently dealing with another issue. Indeed, the operation is not performed on the whole matrix. The threads dont work again after finishing their jobs. For example, I use 512 threads on a vector of size 1000 and only the 512 st values have been changed.



Any idea for this point?

Share Edit Follow **Undelete** Flag

answered Mar 8, 2013 at 14:05



ALFRAM
193 2 14



This is not an answer. Is it? – [KiaMorot](#) Mar 8, 2013 at 14:09

Comments disabled on deleted / locked posts / reviews